North South University

Course name	**:**	**CSE 225**
Section	**:**	**12**
Semester	**:**	**Summer 24**
HW 02 Topic	**:**	**Stack & Queue**
Date	**:**	**25-09-2024**

Submitted To :

Dr. Md. Ahsan Habib (AHbb)

Submitted by :

NAME : Mohammad Tushar Abdullah

ID : 2323766042

HW 03 (PART 01)

ANSWER TO THE QUESTION NO : 01

Q : 01

Qs : What will happen when you attempt to Pop() an element from an empty stack?

Ans : Usually the pop() operation in a stack removes the recently added element. But when a stack is empty that means there is no element in the stack and if we attempt pop() operation it will give ERROR. And according to the given pseudocode,

```
def Pop():
    if top_index == -1:
        print("Stack Underflow") return None
```

it will show “Stack Underflow”.

Qs : If the array has a capacity of 5 and you attempt to push 6 elements, what would happen? Justify your answer based on the pseudocode.

Ans : If the array has capacity of 5 elements and if we want to push 6 elements then it will take first 5 elements and when we will push 6th element it will show an Error and according to the pseudocode ,

```
def Push(key):
    if top_index == capacity - 1:
        print("Stack Overflow")
```

It will show “Stack Overflow”.

Q : 02

Qs : Determine the time complexity of the following operations:

• Push operation • Pop operation • Top operation

Ans : Time complexity for each stack operation is constant. Because we know if we want to do Push(), Pop() or Top() any operation we always do it with the recently added elements that are usually at the end of the array or starting of the linked list. Here the Push() operation directly adds elements at the top index and the Pop() operation removes elements from the top index. Also the Top() operation returns the element of the top index which is constant time complexity. So we don't need to just traverse the whole array or list. In this case time complexity is constant.

Time Complexity :

- Push operation : O(1)
- Pop operation : O(1)
- Top operation : O(1)

Q : 03

Qs: Explain the limitations of using an array for stack implementation, especially in terms of memory allocation.

Ans : When we implement a stack using an array, there are several limitations, especially related to memory allocation. Here’s an explanation of the main limitations:

1. **Fixed Size (Static Memory Allocation):**
   We need to define the size of the array when it is created, meaning the stack has a fixed capacity. If the stack becomes full, we cannot add more elements, leading to a "Stack Overflow." On the other hand, if the

array is too large, we may waste memory because the unused spaces still take up memory.

2. **Stack Overflow:**
   Since the array has a fixed size, adding elements beyond its capacity will cause a "Stack Overflow." If the stack reaches its limit, no more elements can be added, reducing the flexibility of the stack.
3. **Wasted Memory (Over-allocation)**:
   If the maximum size of the stack is set too large, the extra memory remains unused. This leads to inefficient memory usage, especially when only a few elements are stored while the array occupies much more space.
4. **Memory Reallocation and Copying (Dynamic Resizing):**
   If we implement dynamic resizing (like doubling the array size when it is full), we will need to allocate memory for a new, larger array and copy all elements from the old one. This process can be slow and use a lot of memory, increasing time and resource usage in certain situations.
5. **Inefficient for Unpredictable Sizes:**
   If we cannot predict the size of the stack accurately, it becomes difficult to choose the right size for the array. If we allocate too much memory, space is wasted. If we allocate too little, we may face frequent stack overflows or the need for costly resizing.
6. **Non-Dynamic Memory Usage:**
   Arrays don’t allow us to free up memory when we remove elements from the stack. Even if many elements are removed, the allocated memory remains the same, leading to inefficient memory usage, especially when the stack was once large but later became small.

   Using a dynamic data structure like a linked list can avoid many of these problems, as it allows the stack to grow and shrink as needed, without a fixed capacity.

ANSWER TO THE QUESTION NO : 02

Q : 01

Qs : What happens when the stack is empty and you attempt to Pop an element?

Ans : Usually the pop() operation in a stack removes the recently added element. But when a stack is empty that means there is no element in the stack and if we attempt pop() operation it will give ERROR. And according to the given pseudocode,

```
def Pop():
    if self.top is None:
        print("Stack Underflow")
        return None
```

it will show “Stack Underflow”.

Qs : What is the space complexity of this stack implementation for n elements, considering both data and pointers?

Ans : The space complexity of this linked list-based implementation is O(n), where n represents the number of elements. This includes both the data in each node and the pointers that link the nodes together. Each node stores the value of the element and a link to the next one, so as the number of elements increases, the space needed also increases. Therefore, the space used grows in direct proportion to the number of elements.

Q : 02

Qs : Determine the time complexity of the following operations:

• Push operation • Pop operation • Top operation

Ans : Time complexity of the following operations :

Push Operation: O(1)

Only involves adding a new node and updating the top pointer, which are constant time operations.

Pop Operation: O(1)

Simply removes the top element and updates the pointer, with no need to traverse the stack.

Top Operation: O(1)

Accesses the top element directly without modifying the stack or traversing it.

Q : 03

Qs : Compare this linked list-based implementation with the array-based implementation in terms of:

• Memory usage • Flexibility (in terms of stack size) • Performance under different load conditions

Ans : a comparison of linked list-based and array-based implementation :

	Linked List Implementation	Array Implementation
Memory usage	More memory overhead due to storing data and pointers.	Uses less memory, but may have empty spaces or need to grow.

Flexibility (in terms of stack size)	Can easily change size without limits.	Has a fixed size; needs to be resized if full, which takes extra time.
Performance under different load conditions	Works well, but can slow down due to scattered memory use.	Fast for small stacks, but slow down if it needs to resize for large stacks.

ANSWER TO THE QUESTION NO : 03

Scenario: Suppose you have a system that must handle a large and dynamic number of elements, and the system cannot predict the maximum number of elements in advance.

Question: Which stack implementation (Array or Linked List) would you choose for this system? Justify your answer based on memory usage, time complexity, and the need for dynamic sizing.

Ans : In a scenario where a system must handle a large and dynamic number of elements without being able to predict the maximum number in advance, I would choose a linked list-based stack implementation.

Justification:

1. **Memory Usage:** The linked list implementation allows for dynamic memory allocation, using only as much memory as needed for the current elements in the stack. This avoids wasted space that can occur in an array-based implementation, which requires pre-allocating a fixed size. If the array is too small, it may lead to memory inefficiency or overflow issues.

2. **Time Complexity:** Both implementations offer O(1) time complexity for stack operations such as Push, Pop, and Top. However, in an array-based implementation, resizing the array when it reaches capacity can result in an average-case time complexity of O(n) due to the need to copy elements to a new array. In contrast, the linked list maintains O(1) time complexity for all operations consistently.
3. **Dynamic Sizing:** The linked list naturally supports dynamic sizing, allowing nodes to be added or removed without the need for resizing or reallocation. This flexibility is crucial for systems with unpredictable loads, where the number of elements can vary significantly.

HW 03 (PART 02)

ANSWER TO THE QUESTION NO : 01

1. What will happen when you attempt to Dequeue an element from an empty queue?

Ans: When you try to dequeue an element from an empty queue, the condition if size == 0 will be true. The program will print "Queue Underflow" and return None, as there is no element to remove from the queue.

2. If the array has a capacity of 5 and you attempt to add 6 elements, what would happen? Justify your answer based on the pseudocode.

Ans: If the array has a capacity of 5 and you try to add 6 elements, the condition if size == capacity will be true when you attempt to enqueue the 6th element. As a result, the program will print "Queue Overflow" because the queue is full and no more elements can be added beyond its set capacity of 5.

3. Determine the time complexity of the following operations:

- **Enqueue operation**
- **Dequeue operation**
- **Empty operation**

Ans :

- **Enqueue operation:** O(1)

  The enqueue operation simply adds an element to the end of the queue. It involves a single calculation (rear + 1) % capacity and assignment to queue[rear], both of which take constant time.

- **Dequeue operation:** O(1)

  The dequeue operation removes an element from the front of the queue. It involves a calculation (front + 1) % capacity and an assignment to front, both of which take constant time.
- **Empty operation:** O(1)

  Checking if the queue is empty involves comparing the size to 0, which takes constant time.

4. Explain the limitation of using an array for queue implementation, particularly in terms of memory allocation.

Ans: The main limitation of using an array for queue implementation is that the size of the array is fixed when it is created. If the array reaches its full capacity, no more elements can be added unless the array is resized, which is not handled dynamically in a standard array implementation. This can lead to memory inefficiencies:

If the array is too large, it may waste memory.

If the array is too small, it can run out of space, resulting in frequent "Queue Overflow" situations.

ANSWER TO THE QUESTION NO : 02

1. What happens when the queue is empty, and you attempt to Dequeue an element?

Ans: When the queue is empty like front is None, the condition if front is None will be true, and the program will print "Queue Underflow" and return None. This indicates that no element can be removed since the queue is already empty.

2. What is the space complexity of this queue implementation for n elements, considering both data and pointers?

Ans: The space complexity of this queue implementation is O(n). Each node in the queue stores:

One key (data) : O(1)

A pointer to the next node : O(1)

For n elements, the total space complexity is O(n) for both the data and the pointers combined.

3. Determine the time complexity of the following operations:

- **Enqueue operation**
- **Dequeue operation**
- **Empty operation**

Ans:

- **Enqueue operation:** O(1)
  Adding a new node at the rear of the queue is done in constant time because you only need to modify the rear pointer.
- **Dequeue operation:** O(1)
  Removing an element from the front of the queue is also done in constant time by updating the front pointer.
- **Empty operation:** O(1)
  Checking if the queue is empty involves comparing front with None, which takes constant time.

4. Compare this linked list-based queue implementation with the array-based queue implementation in terms of:

- **Memory usage**
- **Flexibility (in terms of queue size)**
- **Performance under different load conditions**

Ans :

Memory usage:

- Array-based queues allocate a fixed amount of memory, which can waste space if the queue isn't full.
- Linked list-based queues allocate memory only as needed, but each node requires extra memory for a pointer.

Flexibility:

- Array-based queues have a fixed size, so they can’t grow without resizing.
- Linked list-based queues are more flexible, growing and shrinking as needed without size limits.

Performance under load:

- Array-based queues may slow down when full and need resizing, wasting memory if too large.
- Linked list-based queues maintain steady performance, but use slightly more memory for pointers.

ANSWER TO THE QUESTION NO : 03

QUESTION : Should you implement a stack or a queue for this system? Justify your answer based on the characteristics of both data structures and how they handle data. Explain the operations that support your decision.

Ans : For processing customer service requests, use a queue because it follows First-In-First-Out (FIFO), ensuring the oldest request is handled first.

A stack uses Last-In-First-Out (LIFO), where the most recent request is handled first, which isn't suitable here.

Key operations:

- **Enqueue:** Adds a request to the end.
- **Dequeue:** Removes the oldest request.

This keeps the process fair and orderly.

ANSWER TO THE QUESTION NO : 04

(a) The queue would work correctly, but Dequeue would take O(n) time if the list were singly-linked with a tail pointer. This is because removing from the back of a singly-linked list requires traversing the entire list, making it inefficient.

(b) Incorrect, because Dequeue would still be O(n) for singly-linked lists.

(c) Incorrect, as Dequeue isn't efficient in a singly-linked list.

(d) Incorrect, the queue would still work, but inefficiently.

(e) Partially correct, but O(n) time is not specified.

(f) Incorrect, doubly-linked lists improve efficiency.